

# The Transformer Architecture

RNN se Complete Transformer Flow tak - A Complete Study Guide

---

## Contents

- 1. RNN & Problems
- 2. Transformer Overview
- 3. Input Embeddings
- 4. Positional Encoding
- 5. Self-Attention (Q, K, V)
- 6. Scaled Dot-Product Attention
- 7. Multi-Head Attention
- 8. Add & Norm
- 9. Feed Forward Network
- 10. Encoder
- 11. Decoder
- 12. Masked Multi-Head Attention
- 13. Cross Attention
- 14. Linear Layer & Softmax
- 15. Teacher Forcing
- 16. Training & Backpropagation
- 17. Inference
- 18. Greedy Search
- 19. Beam Search
- 20. Complete Transformer Flow (Start to End)

# 1. RNN & Problems

## Definition:

Recurrent Neural Network (RNN) ek sequence model hai jo input ko ek-ek token (word) karke, step by step process karta hai. Har step par yeh apni pichli 'hidden state' (yaddasht) ko agle step mein pass karta hai, taake context yaad rahe. Lekin RNN mein bade problems hain: (1) Long-range dependency - lambi sentences mein purani information dheere dheere kho jati hai (vanishing gradient), (2) Sequential processing - har word ko ek ke baad ek process karna parta hai, isliye training slow hoti hai aur GPU parallelization nahi ho pati.

## Example:

Sentence: "The cat, which was sitting on the old wooden chair near the window, was hungry."

RNN ko "was hungry" tak pahunchte pahunchte "cat" ka context kamzor ho jata hai.

Transformer isi problem ko attention mechanism se solve karta hai - har word directly har dusre word ko dekh sakta hai, chahe wo kitna bhi door ho.

# 2. Transformer Overview

## Definition:

Transformer ek neural network architecture hai jo 2017 mein paper "Attention Is All You Need" mein introduce hui. Yeh RNN ki tarah sequential nahi chalta - balke poori sentence ek saath (parallel) process karta hai, aur 'Self-Attention' mechanism use karta hai taake har word directly baaki sab words se apna relationship seekh sake. Iske do main parts hote hain: Encoder aur Decoder.

## Example:

Machine Translation example: "Main school jaa raha hoon" ko English mein "I am going to school" translate karna.

Encoder Urdu sentence ko samajhta hai, Decoder us understanding se English sentence generate karta hai - word by word, lekin poore context ke saath.

# 3. Input Embeddings

## Definition:

Computer sirf numbers samajhta hai, words nahi. Isliye har word (token) ko ek fixed-size vector (numbers ki list) mein convert kiya jata hai, jise Embedding kehte hain. Yeh vector word ka 'meaning' numbers ki form mein represent karta hai - similar meaning wale words ke embeddings bhi ek dusre ke kareeb hote hain vector space mein.

## Example:

Word "king" -> [0.21, -0.45, 0.88, ..., 0.12] (e.g. 512 numbers)

Word "queen" -> [0.19, -0.40, 0.85, ..., 0.15] (king se kareeb hai)

Word "apple" -> [-0.9, 0.3, -0.1, ..., 0.6] (king se bohat door hai)

Yeh embeddings training ke dauran seekhe jate hain.

# 4. Positional Encoding

## Definition:

Transformer poori sentence ek saath (parallel) process karta hai, isliye usay khud pata nahi chalta ke kaunsa word pehle aaya aur kaunsa baad mein. Positional Encoding ek extra vector hai jo har word ke embedding mein add kiya jata hai taake model ko word ki 'position' (order) ka pata chal sake. Yeh sine aur cosine functions se calculate hota hai.

Example:

"Dog bites man" vs "Man bites dog" - dono mein same words hain lekin order alag hai jis se meaning bilkul badal jata hai.  
Position 1 -> PE(1) vector, Position 2 -> PE(2) vector, ... yeh embedding mein add hoke model ko word order ka sense dete hain.

## 5. Self-Attention (Q, K, V)

### Definition:

Self-Attention wo mechanism hai jis se har word sentence ke baaki sab words ko 'dekh' kar decide karta hai ke uske liye kaunsa word zyada important (relevant) hai. Har word ke embedding se 3 alag vectors banaye jate hain: Query (Q) - main kya dhoond raha hoon, Key (K) - mere paas kya information hai, aur Value (V) - actual information jo pass ki jayegi. Q aur K ko compare karke attention scores nikalte hain, jo batate hain kaunsa word kitna dhyan dega kis word par.

Example:

Sentence: "The animal didn't cross the street because it was too tired."  
"it" word ka Query, sentence ke sab words ki Keys se compare hota hai.  
Result: "it" sab se zyada attention "animal" ko deta hai ("street" ko nahi),  
kyunke context se pata chalta hai "it" = "animal".

## 6. Scaled Dot-Product Attention

### Definition:

Yeh Self-Attention ka mathematical formula hai. Query aur Key ka dot product nikala jata hai (similarity check), phir usay Key vector ki dimension ( $d_k$ ) ke square root se divide (scale) kiya jata hai taake numbers bohat bade na ho jayen aur training stable rahe. Phir Softmax lagake yeh scores 0 se 1 ke beech probabilities mein convert hote hain, aur end mein Value vectors ke saath weighted sum nikalta hai.

Example:

Formula:  $\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{Q \times K^T}{\sqrt{d_k}} \right) \times V$   
Agar  $d_k = 64$ , to scores ko  $\sqrt{64}=8$  se divide karenge.  
Softmax ke baad: word "it" ko [animal: 0.7, street: 0.05, tired: 0.15, ...]  
jaisi attention weights milti hain - inka sum V ke saath final output banata hai.

## 7. Multi-Head Attention

### Definition:

Ek single attention sirf ek type ka relationship seekh pata hai. Isliye Transformer Attention ko multiple 'heads' mein split karta hai (e.g. 8 heads) - har head apna alag Q, K, V seekhta hai aur different type ke relationships par focus karta hai (jaise ek head grammar dekhe, dusra meaning, teesra long-distance dependency). End mein sab heads ka output concatenate karke ek final linear layer se pass kiya jata hai.

Example:

Sentence: "The chicken didn't cross the road because it was too wide."  
Head 1: "it" -> "road" (physical meaning ke liye)  
Head 2: "it" -> grammar structure dekhta hai  
Head 3: overall sentence context dekhta hai  
Sab heads mil kar zyada rich understanding dete hain.

## 8. Add & Norm

### Definition:

Yeh do techniques hain jo har sub-layer (Attention ya Feed Forward) ke baad lagayi jaati hain: (1) Add = Residual/Skip Connection - layer ka original input, uske output mein wapis add kar dete hain, taake deep network mein gradient flow smooth rahe aur information loss na ho. (2) Norm = Layer Normalization - values ko normalize karte hain (mean 0, variance 1) taake training stable aur fast rahe.

### Example:

Formula:  $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$

Agar Attention layer ka input  $x$  tha aur output  $\text{Attention}(x)$ , to final result =  $\text{LayerNorm}(x + \text{Attention}(x))$  hoga - original signal bhi preserve rehta hai aur naya learned info bhi shamil ho jata hai.

## 9. Feed Forward Network

### Definition:

Har Encoder aur Decoder block ke andar ek Position-wise Feed Forward Network (FFN) hoti hai, jo har word ki position par independently apply hoti hai (sab positions same weights share karte hain). Yeh do Linear layers hoti hain jinke beech ek non-linear activation (ReLU ya GELU) hoti hai. Iska kaam attention se aayi information ko aur deeply process/transform karna hai.

### Example:

Formula:  $\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$

Agar attention output dimension 512 hai, to FFN andar se usay 2048 dimension tak expand karta hai, phir wapis 512 par le aata hai - isse model complex patterns seekh pata hai.

## 10. Encoder

### Definition:

Encoder input sentence (jaise Urdu sentence) ko padhta hai aur uska rich contextual representation banata hai. Yeh  $N$  identical layers (usually 6) ka stack hota hai, har layer mein: Multi-Head Self-Attention + Add&Norm, phir Feed Forward Network + Add&Norm. Encoder ka final output poori input sentence ki 'understanding' hoti hai, jo Decoder ko di jati hai.

### Example:

Input: "Main school jaa raha hoon"

Encoder Layer 1 -> Layer 2 -> ... -> Layer 6

Output: 5 vectors (har word ke liye ek), har vector mein poori sentence ka context samaya hua hai - yeh Decoder ko "memory" ki tarah pass hota hai.

## 11. Decoder

### Definition:

Decoder output sentence (jaise English translation) ko word-by-word generate karta hai, Encoder ki memory ko use karte hue. Yeh bhi  $N$  layers ka stack hai, har layer mein 3 parts: Masked Multi-Head Self-Attention, Cross-Attention (Encoder output ke saath), aur Feed Forward Network - har ek ke baad Add&Norm.

### Example:

Decoder step-by-step "I", phir "I am", phir "I am going" ... generate karta jata hai jab tak poora sentence "I am going to school" na ban jaye. Har naya word banate waqt, Decoder Encoder ki memory (Urdu sentence ka context) aur ab tak generate hue words dono ko dekhta hai.

## 12. Masked Multi-Head Attention

### Definition:

Decoder mein self-attention ko 'masked' banaya jata hai taake har word sirf apne se pehle wale words ko dekh sake, future (aage wale) words ko nahi - kyunke training/inference ke waqt hum future words 'cheat' nahi kar sakte, unhein predict karna hota hai. Yeh future positions ke scores ko -infinity set karke Softmax se pehle achieve kiya jata hai, taake unki probability 0 ho jaye.

### Example:

Target: "I am going to school"

Word "going" predict karte waqt, model sirf "I am going" tak dekh sakta hai, "to school" nahi dekh sakta - warna model bas answer copy kar lega, kuch seekhega nahi.

## 13. Cross Attention

### Definition:

Cross-Attention Decoder ka wo layer hai jahan Decoder, Encoder ki output (memory) ko dekhta hai. Isme Query Decoder se aati hai (ab tak jo generate kiya), lekin Key aur Value Encoder ke output se aati hain (original input sentence). Isse Decoder decide karta hai ke agla word generate karte waqt input sentence ke kaunse hisse par zyada focus kare.

### Example:

English word "school" generate karte waqt, Cross-Attention Decoder ko batata hai ke Urdu input mein "school" word par sab se zyada focus karo (high attention weight), baaki words par kam.

## 14. Linear Layer & Softmax

### Definition:

Decoder ke sab layers se guzarne ke baad, final output vector ek Linear layer se pass hota hai jo usay poori vocabulary ke size jitna bada vector bana deta hai (e.g. 50,000 words ki vocabulary ho to 50,000 numbers). Phir Softmax function yeh numbers ko probabilities (0 se 1, sum=1) mein convert kar deta hai - jis word ki probability sab se zyada hogi, wahi agla predicted word hoga.

### Example:

Linear layer output: ["school": 8.2, "home": 3.1, "market": 1.5, ...]  
Softmax ke baad: ["school": 0.89, "home": 0.07, "market": 0.02, ...]  
"school" sab se high probability (0.89) rakhta hai, isliye wahi final output word chuna jayega.

## 15. Teacher Forcing

### Definition:

Training ke dauran, Decoder ko agle word predict karne ke liye uska apna pehla predicted output use karne ke bajaye, actual correct (ground truth) previous word diya jata hai. Isse training fast aur stable

hoti hai, kyunke agar model ek galti kare to wo aage sab predictions kharab nahi karti.

**Example:**

Target sentence: "I am going to school"  
"am" predict karte waqt, chahe model ne "I" ke baad kuch aur predict kiya ho, use training mein hamesha real word "I" hi diya jayega - not model ka apna (possibly wrong) guess.

## 16. Training & Backpropagation

### Definition:

Training mein model ka predicted output aur actual correct output ke darmiyan difference (Loss - usually Cross-Entropy Loss) calculate kiya jata hai. Phir Backpropagation algorithm is loss ko network ke har weight tak wapis bhejta hai aur Gradient Descent (ya Adam optimizer) se saare weights thoda thoda adjust kiye jate hain taake agli baar loss kam ho.

**Example:**

Model ne predict kiya "school": probability 0.3 (actual answer),  
Loss zyada hoga kyunke confidence kam thi.  
Backpropagation is error ko Embeddings, Attention, FFN - har jagah wapis bhejta hai aur weights update karta hai taake agli baar "school" ki probability 0.3 se badh kar 0.9 ho jaye.

## 17. Inference

### Definition:

Inference wo stage hai jab trained model ko real use mein laya jata hai (naya, unseen input dena). Yahan Teacher Forcing nahi hoti - Decoder ko apna hi previous predicted output agle step ke liye input ki tarah use karna parta hai, ek word ban jaye to wahi wapis Decoder mein feed hota hai, jab tak special token na aa jaye.

**Example:**

Naya input: "Woh khush hai" (kabhi training mein nahi dekha)  
Step 1: diya -> model ne predict kiya "He"  
Step 2: He -> model ne predict kiya "is"  
Step 3: He is -> "happy" ... yeh chalta rahega jab tak na aaye.

## 18. Greedy Search

### Definition:

Yeh sab se simple decoding strategy hai - har step par model sirf wo ek word chunta hai jiski probability sab se zyada (highest) hoti hai, aur us decision ko wapis check ya badal nahi sakta. Fast hai lekin hamesha best overall sentence nahi deta, kyunke ek locally best choice, overall sentence ko kharab kar sakti hai.

**Example:**

Step 1: "The" (prob 0.9) chuna  
Step 2: "The" ke baad "cat" (prob 0.6) chuna - highest hai isliye  
final: "The cat ..." - lekin ho sakta hai "A feline" jaisa alag path  
overall better translation deta, jo Greedy Search kabhi try hi nahi karta.

## 19. Beam Search

### Definition:

Beam Search, Greedy Search se behtar hai kyunke yeh har step par sirf 1 nahi, balke top-K (beam width) best possible sequences ko parallel track karta hai. Har step ke baad sab beams ko aage expand karke, unme se phir top-K best combinations rakhta hai. End mein jis complete sentence ki overall probability sab se zyada hoti hai, wahi final output banti hai.

### Example:

Beam width = 3 maan lein:

Beam 1: "The cat sat" (score: 0.72)

Beam 2: "A cat sat" (score: 0.68)

Beam 3: "The cats sit" (score: 0.55)

Sab teeno options end tak track hoti hain, phir sab se high overall score wali sentence final answer banti hai.

## 20. Complete Transformer Flow (Start to End)

### In English

1. **Input Sentence** is broken into tokens (words/sub-words).
2. Each token is converted into a numeric **Embedding** vector.
3. **Positional Encoding** is added to embeddings so the model knows word order.
4. These vectors enter the **Encoder** stack (N layers): each layer runs **Multi-Head Self-Attention** (using Q, K, V and Scaled Dot-Product Attention) followed by **Add & Norm**, then a **Feed Forward Network** followed by **Add & Norm** again.
5. The Encoder produces a contextual representation of the whole input sentence, called **Encoder Memory**.
6. On the output side, the target sentence (so far generated) is embedded and positionally encoded, then enters the **Decoder** stack.
7. Inside each Decoder layer: first **Masked Multi-Head Self-Attention** (so the model can't peek at future words) + Add & Norm, then **Cross-Attention** (Query from Decoder, Key/Value from Encoder Memory) + Add & Norm, then a **Feed Forward Network** + Add & Norm.
8. The Decoder's final output passes through a **Linear Layer** that expands it to vocabulary size, then **Softmax** converts it into probabilities for every possible next word.
9. During **Training**, the predicted probabilities are compared to the real target word using a loss function, and **Backpropagation** updates all the weights across the network. **Teacher Forcing** is used so the Decoder always sees the correct previous word during training.
10. During **Inference** (real use), there is no teacher forcing - the Decoder feeds its own previous prediction back in as input for the next step, one word at a time, starting from a <START> token and stopping at an <END> token.
11. The next word at each step is chosen either with **Greedy Search** (pick the single highest-probability word) or **Beam Search** (track several top candidate sequences and pick the best overall sentence at the end).
12. The final output is the complete generated sentence - e.g. a full translation, summary, or answer.

### Roman Urdu / Hinglish Mein

1. **Input sentence** ko chote tokens (words/sub-words) mein tor diya jata hai.
2. Har token ko numbers wale **Embedding** vector mein badal diya jata hai.
3. Embeddings mein **Positional Encoding** add ki jati hai, taake model ko word order (order/sequence) ka pata chal sake.
4. Yeh vectors **Encoder** stack (N layers) mein jate hain: har layer mein pehle **Multi-Head Self-Attention** chalti hai (Q, K, V aur Scaled Dot-Product Attention ke zariye), uske baad **Add & Norm**, phir ek **Feed Forward Network**, uske baad dubara **Add & Norm**.
5. Encoder poori input sentence ka contextual representation banata hai, jise **Encoder Memory** kehte hain.
6. Output side par, ab tak jo target sentence generate hui hai usay bhi embed aur positionally encode kiya jata hai, phir wo **Decoder** stack mein jati hai.
7. Har Decoder layer ke andar: pehle **Masked Multi-Head Self-Attention** (taake model future words na dekh sake) + Add & Norm, phir **Cross-Attention** (Query Decoder se, Key/Value Encoder Memory se) + Add & Norm, phir ek **Feed Forward Network** + Add & Norm.
8. Decoder ka final output ek **Linear Layer** se guzarta hai jo usay poori vocabulary ke size tak expand karta hai, phir **Softmax** in numbers ko har possible agle word ki probability mein badal deta hai.



9. **Training** ke dauran, predicted probabilities ko real (correct) target word se compare kiya jata hai loss function se, aur **Backpropagation** poore network ke weights ko update karti hai. **Teacher Forcing** use hoti hai taake Decoder ko training mein hamesha sahi previous word diya jaye.

10. **Inference** (asal use) ke waqt teacher forcing nahi hoti - Decoder apna hi pichla prediction wapis agle step ke input ki tarah use karta hai, ek ek word karke, <START> token se shuru hoke <END> token par ruk jata hai.

11. Har step par agla word ya to **Greedy Search** se chuna jata hai (sirf sab se high probability wala word), ya **Beam Search** se (kai top candidate sentences ko track karke end mein sab se best overall sentence chuni jati hai).

12. Final output poori generated sentence hoti hai - jaise ek complete translation, summary, ya jawab.